

COMP 3311: Database Management Systems

Query Processing Cost Estimates (page I/Os)

1. SELECTION: Equality Search

Algorithm A1 (linear search)

B the number of pages that contain records of relation r

or

$B/2$ if selection is on a key attribute since we stop when we find the record (as there is at most only one with the specified value).

Algorithm A2 (binary search)

$\lceil \log_2(B) \rceil$ if equality selection is on a candidate key attribute; otherwise,

$\lceil \log_2(B) \rceil$ **plus** the number of pages that contain records that satisfy the selection condition.

Algorithm A3 (using a clustering index)

B+-tree index on candidate key: $HT_i + 1$ where HT_i is the height of the tree index

B+-tree index on non-candidate key: HT_i + the number of pages that contain records that satisfy the selection condition

Algorithm A4 (using a non-clustering (secondary) index)

Search key is a candidate key

B+-tree index: $HT_i + 1$ where HT_i is the height of the tree index

Hash index: $1 + 1$ or $1.2 + 1$ if there exist overflow buckets

Search key is non-candidate key

B+-tree index: HT_i + cost to retrieve indirection pointers + number of records retrieved (assumes each record requires a page access)

Hash index: 1 + cost to retrieve indirection pointers + number of records retrieved or 1.2 + cost to retrieve indirection pointers + number of records retrieved if there exist overflow buckets

2. SELECTION: Range Search

Algorithm A5 (using a clustering B+-tree index, relation sorted on A)

$\sigma_{A \geq v}(r)$ use the index to find the first tuple where $A \geq v$ and scan the relation sequentially from there.

$\sigma_{A \leq v}(r)$ do not use the index; instead scan the relation sequentially until you find the first tuple where $A > v$.

Algorithm A6 (using a non-clustering (secondary) B+-tree index)

$\sigma_{A \geq v}(r)$ use the index to find first index entry where $A \geq v$ and scan the index sequentially from there to find pointers to the tuples.

$\sigma_{A \leq v}(r)$ scan the leaf pages of the index finding pointers to records until the first entry where $A > v$.

3. SELECTION: Complex Predicates

Algorithm A7 (conjunction using one B+-tree index)

Select a combination of θ_i and algorithms A1 through A7 that results in the least cost for $\sigma_{\theta_i}(r)$. Check the remaining conditions on the tuple after retrieving it into memory.

Algorithm A8 (conjunctive using a composite B⁺-tree index)

If available, use an appropriate composite (multiple-attribute) index.

The type of index determines which of the algorithms A2, A3 or A4 will be used.

Algorithm A9 (conjunctive selection by intersection of identifiers)

Using indexes: sum of the costs of the individual index scans **plus** the cost of retrieving the tuples

Algorithm A10 (disjunctive selection by union of identifiers)

Using indexes: sum of the costs of the individual index scans **plus** the cost of retrieving the tuples

4. SELECTION: Negation (NOT)

Use linear search on the file.

If very few records satisfy the negation and an index is applicable for the condition, then find satisfying records using the index and retrieve the records from the file.

5. JOIN

Block nested-loop join

Worst case: $B_r * B_s + B_r$

Best case: $B_r + B_s$

If neither relation fits in memory, use the smaller relation as the outer relation r . If a relation fits in memory, use it as the inner relation s .

Indexed Nested-Loop Join

$B_r + n_r * c$ where c is the cost of traversing the index and fetching all matching s tuples for one tuple or r .

Merge Join

$B_r + B_s + \text{cost of sorting if relations are unsorted}$

Hash Join

$3 * (B_r + B_s)$

6. PROJECTION

Using External Sorting

Pass 0: the original relation is read, and the same number of smaller tuples are written out.

Merge passes: fewer tuples are written out in each pass.

7. AGGREGATE OPERATIONS

Without Grouping

In general, requires scanning the relation.

If there is a B⁺-tree index whose search key includes *all attributes* in the **select** or **where** clauses, then can do an index-only scan.

With Grouping

Can use the same techniques as for duplicate elimination.

Instead of eliminating duplicates, gather them into groups and apply aggregate operations on each group.

If there is a B⁺-tree index whose search key includes *all attributes* in **select**, **where** and **group by** clauses, then can do an index-only scan.